

Multi-Model AI Chatbot - Production Grade Documentation

FastRouter.ai Integration | Token & Cost Optimization Platform

Version: 1.0.0

Last Updated: June 13, 2026

Author: Product Engineering Team

Status: Production Ready

DOCUMENTATION INDEX

#	Document	Purpose	Audience
1	PRD	Product Requirements Document	PMs, Stakeholders
2	Architecture Design	System architecture & data flow	Architects, Leads
3	API Design	REST/WS API contracts	Backend Engineers
4	Database Schema	Data models & relationships	Backend Engineers
5	Model Routing Engine	Routing logic & algorithms	ML Engineers
6	Cost Optimization Spec	Token budgeting & controls	DevOps, Finance
7	Security & Compliance	Auth, encryption, PII	Security Engineers
8	Deployment Guide	Infrastructure & CI/CD	DevOps Engineers
9	Monitoring & Observability	Metrics, logging, alerting	SRE Team

#	Document	Purpose	Audience
10	Testing Strategy	Test plans & QA	QA Engineers
11	Frontend Spec	UI/UX component design	Frontend Engineers
12	Runbook	Incident response & troubleshooting	On-call Engineers

1. PRODUCT REQUIREMENTS DOCUMENT (PRD)

1.1 Product Vision

Build an intelligent multi-model chatbot that optimizes AI costs by 70-85% through intelligent model routing while maintaining GPT-4 class quality. Users can either let the system auto-route prompts to optimal models or manually assign tasks to specific models using @mentions.

1.2 Core Features

Feature 1: Auto-Optimize Mode (Smart Routing)

User Story: As a user, I want the system to automatically choose the best model(s) for my task so I don't have to think about which AI to use.

Acceptance Criteria: - System analyzes prompt complexity (length, domain, reasoning depth, code presence) - Routes to single model for simple tasks (<100 words, factual queries) - Routes to multiple models in parallel for complex tasks (reasoning, code generation, multi-step) - Aggregates responses when multiple models are used - Provides transparency: shows which model(s) were used and estimated cost - Fallback chain: if primary model fails, auto-switch to backup within 500ms

Cost Target: 50-85% reduction vs using GPT-4o for all queries

Feature 2: Manual Assignment Mode (@Mentions)

User Story: As a power user, I want to assign specific sub-tasks to specific models using @mentions for precise control.

Acceptance Criteria: - Parse @model_name tags in prompts (e.g., @claude-sonnet, @gpt-4o) - Support multiple @mentions in single prompt - Parallel

execution of tagged sub-tasks - Sequential dependency: @model2 after @model1 completes - Context passing: output from @model1 feeds into @model2 - Cost preview before execution: "This will cost ~\$0.023 and use 3 models"

Example Prompts:

@claude-sonnet Write a Python function to calculate Fibonacci numbers with memoization.

@gpt-4o-mini Write unit tests for the above function covering edge cases.

@gemini-flash Generate documentation for this function.

Feature 3: Token & Cost Dashboard

User Story: As a user, I want to see real-time cost analytics so I can optimize my spending.

Acceptance Criteria: - Real-time cost tracking per request, per model, per day -

Cost comparison: "You saved \$12.40 this week using smart routing" - Token usage breakdown: input vs output tokens - Budget alerts at 50%, 75%, 90% of \$49 credit - Usage forecasting: "At current rate, credits will last 18 days"

1.3 Success Metrics (KPIs)

Metric	Target	Measurement
Cost per 1K requests	<\$0.50	Via FastRouter analytics
Routing accuracy	>95%	Human evaluation on test set
Avg response time	<2s	95th percentile
User satisfaction	>4.5/5	Post-session survey
Credit utilization	30 days	Time to deplete \$49
Fallback rate	<2%	Auto-failover events

1.4 User Personas

1. **Casual User** - Wants answers, doesn't care about models. Uses Auto mode.
2. **Developer** - Needs code generation, testing, documentation. Uses @mentions.
3. **Cost-Conscious Startup** - Wants enterprise quality at startup